Ejercicios de Autoevaluación (Tipo Test)

Tema 07: Preprocesador y Compilación Separada

Informática Industrial y Comunicaciones — Universidad de Oviedo

---

1. ¿Qué valor imprimirá este programa por pantalla tras ser procesado y ejecutado?

```
#include <stdio.h>

#define DOBLE(x) x + x

int main(void) {
    int resultado = DOBLE(3) * DOBLE(3);
    printf("%d\n", resultado);
    return 0;
}
```

a) 36

b) 15 ✓ (Opción Correcta)

c) 12

d) Error de compilación por falta de paréntesis en la macro.

Respuesta correcta: b) 15

El preprocesador realiza una sustitución textual directa. La expresión se expande a `3 + 3 * 3 + 3`. Por la precedencia de operadores en C, se evalúa la multiplicación antes que las sumas: $3 + (3 \cdot 3) + 3 = 15$.

2. Analiza la estructura del siguiente proyecto compuesto por tres archivos (`motor.h`, `uno.c` y `dos.c`):

```
// ----- Archivo: motor.h -----
#ifndef MOTOR_H
#define MOTOR_H
int estado_motor = 0; // 0 = Apagado, 1 = Encendido
#endif

// ----- Archivo: uno.c -----
#include "motor.h"
// Código del módulo uno.c...

// ----- Archivo: dos.c -----
#include "motor.h"
// Código del módulo dos.c...
```

a) Compilará y enlazará perfectamente gracias a la protección del `#ifndef MOTOR_H`.

b) Dará un error en tiempo de compilación dentro de `dos.c` diciendo que `estado_motor` no está definido.

c) Compilará cada .c por separado, pero el enlazador (linker) fallará con un error de múltiple definición ✓ (Opción Correcta)

d) Ninguna de las anteriores.

Respuesta correcta: c) Error de enlazado (multiple definition)
Escribir `int estado_motor = 0;` en un `.h` es una *definición* que reserva memoria RAM. Al incluirlo en `uno.c` y `dos.c`, ambos generan archivos objeto reclamando esa misma variable en memoria. Los *include guards* evitan duplicados dentro del mismo `.c`, pero no entre distintos archivos `.c`.

3. Observa el siguiente código y determina qué valor almacenará la variable `res`:

```
#include <stdio.h>

#define RESTA(a, b) a - b

int main(void) {
    int res = RESTA(10, 5 - 2);
    printf("%d\n", res);
    return 0;
}
```

a) 7 (resultado de 10 − 3)

b) 3 ✓ (Opción Correcta)

c) 13

d) Error de sintaxis en la llamada a la macro.

Respuesta correcta: b) 3

Al expandir literalmente los parámetros, la llamada `RESTA(10, 5 - 2)` se convierte en `10 - 5 - 2`. Evaluando de izquierda a derecha: $10 - 5 = 5$, y $5 - 2 = 3$. Para que funcionara como se espera ($10 - (5-2) = 7$), la definición tendría que ser `#define RESTA(a, b) ((a) - (b))`.

4. Dado el siguiente proyecto para controlar un temporizador:

```
// --- temporizador.h ---
#ifndef TEMPORIZADOR_H
#define TEMPORIZADOR_H
extern int ms_transcurridos;
void Tick(void);
#endif

// --- temporizador.c ---
#include "temporizador.h"
int ms_transcurridos = 0;
void Tick(void) { ms_transcurridos++; }

// --- main.c ---
#include <stdio.h>
#include "temporizador.h"
int main(void) {
    Tick();
    printf("%d\n", ms_transcurridos);
    return 0;
}
```

a) Fallará en la compilación de `main.c` porque no sabe qué tipo de dato es `ms_transcurridos`.

b) Dará error de enlazado porque `ms_transcurridos` se declara dos veces.

c) Compilará y enlazará correctamente, imprimiendo el valor 1 por pantalla ✓ (Opción Correcta)

d) Dará error de compilación porque `extern` solo se puede usar dentro de archivos `.c` y nunca en un `.h`.

Respuesta correcta: c) Compilará y enlazará correctamente

Es la metodología correcta de compilación separada en C. La memoria física se asigna una sola vez en `temporizador.c`, mientras que la cabecera `.h` solo utiliza `extern` para notificar a los demás módulos de la existencia del símbolo sin duplicar RAM.

5. Considera el siguiente programa con directivas del preprocesador:

```
#include <stdio.h>

#define MODO_DEPURACION 0

int main(void) {
    int x = 50;

#if MODO_DEPURACION > 0
    printf("Modo depuracion activo. x = %d\n", x);
#else
    printf("Modo produccion activo.\n");
#endif

    return 0;
}
```

a) El compilador incluirá las dos instrucciones `printf` en el ejecutable final y la condición se evaluará en tiempo de ejecución.

b) El preprocesador eliminará el primer printf antes de la compilación, dejando solo el mensaje de producción ✓ (Opción Correcta)

c) Ocurrirá un error de compilación porque `#if` no puede comparar con el número 0.

d) Imprimirá ambos mensajes por pantalla.

Respuesta correcta: b) El preprocesador elimina el bloque inactivo

La compilación condicional (`#if / #else / #endif`) la resuelve el preprocesador antes del compilado. Como `MODO_DEPURACION > 0` es falso ($0 > 0$ es 0), la primera rama se descarta textualmente y no ocupa espacio en el ejecutable final.

6. Observa la estructura del siguiente proyecto dividido en tres archivos (`utilidades.h`, `uno.c` y `dos.c`):

```
// ----- Archivo: utilidades.h -----
#ifndef UTILIDADES_H
#define UTILIDADES_H

void ImprimirMensaje(void) {
    printf("Hola desde utilidades\n");
}

#endif

// ----- Archivo: uno.c -----
#include "utilidades.h"
// Código del módulo uno.c...

// ----- Archivo: dos.c -----
#include "utilidades.h"
// Código del módulo dos.c...
```

a) Funciona correctamente porque los include guards impiden que la función se duplique.

b) El preprocesador da un error al leer la llave `{` dentro de un archivo de cabecera.

c) El enlazador (linker) falla con un error de función duplicada (multiple definition of 'ImprimirMensaje') ✓ (Opción Correcta)

d) Se ejecuta dos veces la función al iniciar el programa.

Respuesta correcta: c) Error de enlazado (multiple definition)

Escribir el cuerpo `{ ... }` de una función dentro de un `.h` es una *definición*. Al incluir la cabecera en `uno.c` y `dos.c`, el preprocesador copia el cuerpo en ambos archivos, haciendo que `uno.obj` y `dos.obj` contengan el mismo código ejecutable. El enlazador detecta el duplicado y lanza un fallo.

Práctica de Programación Modular: Módulo de Geometría

Diseñar un programa en C estructurado en tres archivos (`geometria.h` , `geometria.c` y `main.c`):

- **`geometria.h` :** Interfaz pública del módulo. Debe incluir las **guardas de inclusión** (Include Guards), definir la constante `PI` (3.14159), una macro parametrizada `CUADRADO(x)` libre de efectos secundarios, y los prototipos para calcular el **área** y el **perímetro** de un círculo dado su radio.
- **`geometria.c` :** Implementará la lógica ejecutable de las funciones declaradas en la cabecera.
- **`main.c` :** Solicitará un radio al usuario por teclado e imprimirá el área y el perímetro calculados. Además, definirá una macro de configuración `MODO_DEPURACION` y, mediante **compilación condicional**, mostrará un mensaje adicional con el resultado de la macro `CUADRADO` únicamente si la depuración está activa (> 0).

---

1. Archivo: `geometria.h`

```
#ifndef GEOMETRIA_H
#define GEOMETRIA_H

// Constante simbólica de Pi
#define PI 3.14159f

// Macro con paréntesis preventivos en parámetros y en la expresión global
#define CUADRADO(x) ((x) * (x))

// Prototipos de las funciones públicas del módulo
float CalcularAreaCirculo(float radio);
float CalcularPerimetroCirculo(float radio);

#endif // GEOMETRIA_H
```

2. Archivo: `geometria.c`

```
#include "geometria.h"

// Cálculo del área: PI * r^2
float CalcularAreaCirculo(float radio) {
    return PI * CUADRADO(radio);
}

// Cálculo del perímetro: 2 * PI * r
float CalcularPerimetroCirculo(float radio) {
    return 2.0f * PI * radio;
}
```

3. Archivo: `main.c`

```
#include <stdio.h>
#include "geometria.h"

// Configuración de compilación condicional
#define MODO_DEPURACION 1
```

```c
int main(void) {
    float radio;

    printf("Introduzca el radio del circulo: ");
    if (scanf("%f", &radio) != 1 || radio < 0.0f) {
        printf("Error: Radio invalido.\n");
        return 1;
    }

    // Invocación a las funciones del módulo
    float area = CalcularAreaCirculo(radio);
    float perimetro = CalcularPerimetroCirculo(radio);

    printf("\n--- RESULTADOS ---\n");
    printf("Area del circulo     : %.2f\n", area);
    printf("Perimetro del circulo: %.2f\n", perimetro);

    // Compilación condicional evaluada en tiempo de preprocesado
#if MODO_DEPURACION > 0
    printf("\n[DEBUG] Evaluacion de la macro CUADRADO(%.2f) = %.2f\n",
           radio, CUADRADO(radio));
#endif

    return 0;
}
```